APPLE FIREPRO D500 GPU PASSTHROUGH ON PROXMOX VE

A Tested Troubleshooting and Automatic Recovery Guide for the 2013 Mac Pro 6,1

VFIO • Apple VFCT/VBIOS • QMP Verification • Cold-Boot Recovery

Validated configuration

Proxmox VE 9.2.21 • Linux 7.0.14-20-pve • Ubuntu Server 24.04

AMD FirePro D500 (Tahiti) • Jellyfin VM

Community Technical Guide • Revision 1.0 • October 2026

Scope, Disclaimer, and Safety

This guide documents a working solution developed on one Apple Mac Pro 6,1 with dual AMD FirePro D500 GPUs. It is intended to help other Proxmox users reproduce the diagnostic method and adapt the recovery mechanism to their own hardware. It is not a guarantee that every Mac Pro, firmware revision, kernel, or D500 will behave identically.

> **WARNING:** *Back up the Proxmox host and affected VM before changing VFIO, initramfs, PCI passthrough, or systemd startup behavior. PCI addresses and IOMMU groups are machine-specific. Verify yours before copying any command.*
>
> **WARNING:** *This procedure does NOT flash the GPU. Do not flash the extracted ROM, do not use an unverified ROM from the Internet, and do not force PCI bridge resets with raw configuration writes. The ROM is supplied to QEMU only.*
>
> **NOTE:** *The Apple/AMD ROM binaries and VFCT table are intentionally not distributed with this guide. Extract and validate firmware from your own hardware. Reference hashes from the test system are included only as reproducibility data.*

Executive Summary

The core failure was not simply “GPU passthrough does not work.” The D500 could be correctly bound to vfio-pci, wake to D0, and initially appear inside QEMU, yet still disappear several seconds after VM startup. A normal qm start could therefore succeed while the guest ultimately booted without the GPU. A single early QMP check could also produce a false positive.

The reliable solution combines correct early VFIO ownership, explicit PCI power-state preparation, the correct Apple VBIOS extracted from ACPI VFCT, QEMU PCI topology verification, and a controlled two-attempt startup procedure. The final helper checks QEMU every five seconds for 30 seconds. If the D500 disappears, the helper stops the failed VM, allows VFIO to reclaim the device, wakes it again, and performs one retry.

- Cold-boot D500 state observed: vfio-pci + D3hot.
- First assignment can fail even after initially appearing successful.
- The physical GPU may temporarily disappear from the PCI bus.
- The second wake + assignment repeatedly succeeded in testing.
- Final unattended cold-boot validation automatically caught a delayed dropout and recovered on attempt 2.
- Guest validation confirmed radeon, /dev/dri/renderD128, UVD, and VCE.

Test Platform

Component	Validated configuration
Host	Apple Mac Pro 6,1 (Late 2013 cylinder)
Hypervisor	Proxmox VE 9.2.21
Kernel	7.0.14-20-pve

Boot	EFI
IOMMU	Intel IOMMU
Guest	Ubuntu Server 24.04
VM	103, q35, CPU host, 6 cores, 8424 MB RAM
Application	Jellyfin
Passed GPU	FirePro D500 at 06:00.0
GPU audio	06:00.1

D500 PCI identities on the test host

PCI address	Function	PCI ID	Apple subsystem
02:00.0	FirePro D500 GPU	1002:679e	106b:0126
06:00.0	FirePro D500 GPU	1002:679e	106b:0125
06:00.1	Tahiti HDMI audio	1002:aaa0	N/A

> **NOTE:** *Both GPUs share AMD device ID 1002:679e. Matching only on the basic GPU ID does not distinguish the two physical D500s.*

1. Identify the Correct Devices

Start by inventorying the host. Do not assume that another Mac Pro will use the same bus addresses.

```
lspci -nnk | grep -A4 -Ei 'VGA|Display|Audio'

lspci -nnvv -s 06:00.0
lspci -nnvv -s 06:00.1
```

Record the GPU and audio PCI addresses, vendor/device IDs, subsystem IDs, IOMMU groups, and current drivers.

```
readlink -f /sys/bus/pci/devices/0000:06:00.0/iommu_group
readlink -f /sys/bus/pci/devices/0000:06:00.1/iommu_group
```

2. Configure IOMMU and VFIO

IOMMU

The validated host used the following kernel command line:

```
intel_iommu=on iommu=pt pcie_acs_override=downstream,malfunction nofb nomodeset video=vesafb:off,efifb:off
```

> **WARNING:** *Do not copy the ACS override parameter merely because it appears here. ACS override can change apparent IOMMU grouping without creating true hardware isolation. Start with the minimum IOMMU configuration your system needs.*

```
dmesg | grep -Ei 'DMAR|IOMMU'
```

VFIO binding

The working host reserved the relevant devices for vfio-pci during early boot. The test system contained this line:

```
options vfio-pci ids=1002:679e,8086:1d20,1002:aaa0 disable_vga=1
```

> **NOTE:** *8086:1d20 was another device intentionally reserved on the test host. It is not part of the D500. Do not add it unless it is appropriate for your machine. A D500-focused starting point is:*

```
options vfio-pci ids=1002:679e,1002:aaa0 disable_vga=1
```

> **WARNING:** *Because both D500 GPUs use 1002:679e, binding by PCI ID can reserve both. If one D500 must remain with the host, use a more selective binding strategy.*

A real troubleshooting bug was caused by spaces after commas in the IDs list. vfio-pci interpreted later IDs as parameters and ignored them. Use commas with no spaces.

```
vfio
vfio_iommu_type1
vfio_pci
```

Place those module names in /etc/initramfs-tools/modules, then rebuild the current initramfs:

```
update-initramfs -u -k "$(uname -r)"
```

After rebooting, verify both functions are owned by vfio-pci before proceeding.

```
lspci -nnk -s 06:00.0
lspci -nnk -s 06:00.1
```

3. Understand the D3hot Power-State Problem

On a pristine cold boot, the selected D500 was repeatedly observed in D3hot while already bound to vfio-pci. Correct driver ownership therefore did not imply that the GPU was ready for assignment.

```
cat /sys/bus/pci/devices/0000:06:00.0/power_state
readlink -f /sys/bus/pci/devices/0000:06:00.0/driver
```

The working preparation step forces runtime power control to on for both functions, then allows two seconds for the device to reach D0:

```
echo on > /sys/bus/pci/devices/0000:06:00.0/power/control
echo on > /sys/bus/pci/devices/0000:06:00.1/power/control
sleep 2
cat /sys/bus/pci/devices/0000:06:00.0/power_state
cat /sys/bus/pci/devices/0000:06:00.1/power_state
```

4. Understand the Reset Limitation

Proxmox frequently logged the following when assigning the D500:

```
error writing '1' to '/sys/bus/pci/devices/0000:06:00.0/reset': Inappropriate ioctl for device
failed to reset PCI device '0000:06:00.0', but trying to continue as not all devices need a reset
```

The GPU advertised the bus reset method while the audio function advertised pm:

```
cat /sys/bus/pci/devices/0000:06:00.0/reset_method
# observed: bus
```

```
cat /sys/bus/pci/devices/0000:06:00.1/reset_method
# observed: pm
```

A direct write to the GPU reset node failed with the same ioctl error. The upstream root port (00:03.0 on this host) did not expose a usable sysfs reset node. The final solution therefore does not depend on forcing a hardware reset.

> **WARNING:** *Do not use setpci or raw Bridge Control reset-bit manipulation as a casual workaround. A bridge reset can affect other devices and destabilize the host.*

5. Recover the Correct Apple VBIOS from ACPI VFCT

The ordinary PCI expansion-ROM path was not usable on this machine. Even after native Linux initialized the D500 with radeon, reading the ROM through sysfs produced an I/O error and an invalid 0x0000 ROM signature. The useful firmware source was Apple's ACPI Video Firmware Configuration Table (VFCT).

Acquire VFCT from your own Mac Pro

```
ls /sys/firmware/acpi/tables/

# If VFCT is present:
cp /sys/firmware/acpi/tables/VFCT /root/VFCT.bin
sha256sum /root/VFCT.bin
```

If VFCT is not exposed under the hypervisor, boot a native Linux live environment. Ubuntu 24.04.5 Live was used successfully during the investigation.

What was found on the validated machine

The test VFCT was 131,232 bytes and contained two valid 65,536-byte AMD option ROMs. Their board strings and physical mapping were:

ROM	Physical GPU	Apple subsystem	SHA-256
D500-LA-029.rom	02:00.0	106b:0126	eb2aec73a30e93aa3867a34e4eb2eeca01c421780fff3dd5ca727ffa53227a99
D500-LB-029.rom	06:00.0	106b:0125	c07040282180be27bc795c8a8e4f579b9dd3bee20ef3cd0dc025d7fec9675d31

The VFCT itself had SHA-256:

```
267ae5cd39050b2ffdd0d5911c3e860aa8fb5dce745c7dfc2e9aecb367fe65a8
```

The two option ROMs began at offsets 0x00068 and 0x10084 in this specific table. The 28 bytes preceding each ROM contained information that allowed the ROMs to be associated with PCI bus 02 and bus 06 respectively.

> **WARNING:** *These offsets and hashes are reference data from one Mac Pro. Do not blindly split another VFCT at the same offsets or assume the hashes must match. Parse and validate your own table.*

> **NOTE:** *For a public GitHub repository, publish the extraction/validation method and reference hashes, not the extracted ROM or VFCT binaries. This avoids redistributing Apple/AMD firmware and reduces the risk that users feed QEMU a firmware image from the wrong board revision.*

6. Install the Device-Specific ROM for QEMU

The Jellyfin VM used the physical D500 at 06:00.0, so the matching LB firmware was installed as:

```
/usr/share/kvm/D500-LB-029.rom
```

The relevant Proxmox VM line was:

```
hostpci0: 06:00,pcie=1,romfile=D500-LB-029.rom
```

Inspect the generated QEMU command to confirm both functions and the ROM path:

```
qm showcmd 103 --pretty
```

On the validated VM, QEMU created hostpci0.0 for the GPU and hostpci0.1 for its audio function. Those QEMU device IDs are what the recovery helper verifies.

7. The Failure That Makes This Problem Deceptive

The decisive discovery was that VM process state and GPU assignment state are different things. QEMU could be running while the D500 was missing. Worse, the D500 could appear initially and disappear several seconds later.

Three observed cold-boot failure variants

1. Physical disappearance: the D500 functions vanished from host PCI/sysfs for several seconds, re-enumerated under vfio-pci, and left the first QEMU instance running with an empty downstream bus.
2. Empty QEMU bus without an observed host disappearance: the physical functions remained visible, but QEMU's PCIe bridge contained no D500.
3. Delayed dropout: QMP initially showed hostpci0.0 and hostpci0.1, so a five-second verification passed. The devices later disappeared from QEMU before Ubuntu enumerated them.

> **NOTE:** *The third variant is why a single "GPU is present" check immediately after startup is not enough.*

Inspect QEMU directly

```
qm monitor 103

# At the QEMU monitor prompt:
info pci
```

A successful assignment showed Bus 1, device 0, function 0 as AMD 1002:679e with id hostpci0.0 and function 1 as AMD 1002:aaa0 with id hostpci0.1. Failed starts left the PCIe bridge present but Bus 1 empty.

8. Final Automatic Recovery Script (V3)

The helper below encodes the successful manual recovery sequence. It validates vfio-pci ownership, wakes both functions to D0, starts the VM, then checks QEMU every five seconds for 30 seconds. A failed first attempt is stopped, the host state is allowed to settle, and one second attempt is made. There is no endless retry loop.

> **WARNING:** *The recovery path uses qm stop. During this startup failure the guest had not reached a useful operating state, but qm stop is still a hard stop. Adapt the policy if your VM can perform meaningful writes before the failure is detected.*

```
#!/bin/bash

set -u

VMID=103

GPU="0000:06:00.0"
AUDIO="0000:06:00.1"

GPU_PATH="/sys/bus/pci/devices/$GPU"
AUDIO_PATH="/sys/bus/pci/devices/$AUDIO"

QMP_SOCKET="/run/qemu-server/${VMID}.qmp"

TAG="d500-jellyfin"

log() {
  echo "$*"
  logger -t "$TAG" -- "$*"
}

fail() {
  log "ERROR: $*"
  exit 1
}

device_present() {
  [ -e "$1" ]
}

driver_name() {
  basename "$(readlink -f "$1/driver" 2>/dev/null)" 2>/dev/null
}

verify_vfio() {
  local gpu_driver
  local audio_driver

  device_present "$GPU_PATH" ||
    fail "$GPU is not present"

  device_present "$AUDIO_PATH" ||
    fail "$AUDIO is not present"

  gpu_driver="$(driver_name "$GPU_PATH")"
  audio_driver="$(driver_name "$AUDIO_PATH")"

  log "Drivers: GPU=$gpu_driver AUDIO=$audio_driver"
```

```
    [ "$gpu_driver" = "vfio-pci" ] ||
        fail "$GPU is not bound to vfio-pci"

    [ "$audio_driver" = "vfio-pci" ] ||
        fail "$AUDIO is not bound to vfio-pci"
}

wake_d500() {
    device_present "$GPU_PATH" ||
        fail "$GPU is not present"

    device_present "$AUDIO_PATH" ||
        fail "$AUDIO is not present"

    log "Waking D500 GPU and audio functions"

    echo on > "$GPU_PATH/power/control" ||
        fail "Could not wake $GPU"

    echo on > "$AUDIO_PATH/power/control" ||
        fail "Could not wake $AUDIO"

    sleep 2

    local gpu_power
    local audio_power

    gpu_power="$(cat "$GPU_PATH/power_state" 2>/dev/null)"
    audio_power="$(cat "$AUDIO_PATH/power_state" 2>/dev/null)"

    log "Power state after wake: GPU=$gpu_power AUDIO=$audio_power"

    [ "$gpu_power" = "D0" ] ||
        fail "$GPU did not reach D0"

    [ "$audio_power" = "D0" ] ||
        fail "$AUDIO did not reach D0"
}

wait_for_vm_stopped() {
    local timeout=30
    local elapsed=0

    while [ "$elapsed" -lt "$timeout" ]; do
        if qm status "$VMID" 2>/dev/null | grep -q "status: stopped"; then
            return 0
        fi
        sleep 1
        elapsed=$((elapsed + 1))
    done
    return 1
}

wait_for_qmp() {
    local timeout=30
    local elapsed=0

    while [ "$elapsed" -lt "$timeout" ]; do
        if [ -S "$QMP_SOCKET" ]; then
            return 0
        fi
        sleep 1
        elapsed=$((elapsed + 1))
```

```
    done
    return 1
}

verify_qemu_d500() {
    python3 - "$QMP_SOCKET" <<'PYQMP'
import json
import socket
import sys

socket_path = sys.argv[1]
wanted = {"hostpci0.0", "hostpci0.1"}

def find_qdev_ids(obj):
    found = set()

    if isinstance(obj, dict):
        qdev = obj.get("qdev_id")

        if qdev in wanted:
            found.add(qdev)

        for value in obj.values():
            found |= find_qdev_ids(value)

    elif isinstance(obj, list):
        for item in obj:
            found |= find_qdev_ids(item)

    return found

try:
    sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    sock.settimeout(5)
    sock.connect(socket_path)

    f = sock.makefile("rwb", buffering=0)

    json.loads(f.readline())

    f.write(b'{"execute":"qmp_capabilities"}\n')
    capabilities = json.loads(f.readline())

    if "error" in capabilities:
        sys.exit(1)

    f.write(b'{"execute":"query-pci"}\n')
    response = json.loads(f.readline())

    if "error" in response:
        sys.exit(1)

    found = find_qdev_ids(response)

    if found == wanted:
        sys.exit(0)

    sys.exit(1)

except Exception:
    sys.exit(1)
PYQMP
}
```

```
start_and_verify() {
  local attempt="$1"
  local check
  local checks=6
  local interval=5

  log "Starting VM $VMID: attempt $attempt"

  qm start "$VMID" ||
    return 1

  if ! wait_for_qmp; then
    log "QMP socket did not appear on attempt $attempt"
    return 1
  fi

  log "Beginning D500 stabilization check on attempt $attempt"

  for check in $(seq 1 "$checks"); do
    sleep "$interval"

    if verify_qemu_d500; then
      log "D500 stabilization check $check/$checks PASS"
    else
      log "D500 stabilization check $check/$checks FAIL"
      return 1
    fi
  done

  log "D500 remained attached for $((checks * interval)) seconds on attempt $attempt"
  return 0
}

log "Starting D500-aware startup for VM $VMID"

if qm status "$VMID" 2>/dev/null | grep -q "status: running"; then
  log "VM $VMID is already running"

  if [ -S "$QMP_SOCKET" ] && verify_qemu_d500; then
    log "D500 is already attached correctly; nothing to do"
    exit 0
  fi

  fail "VM $VMID is already running but D500 QMP verification failed"
fi

verify_vfio
wake_d500

if start_and_verify 1; then
  log "SUCCESS: VM $VMID started with D500 on first attempt"
  exit 0
fi

log "First attempt did not acquire D500; stopping VM $VMID"

if qm status "$VMID" 2>/dev/null | grep -q "status: running"; then
  qm stop "$VMID" ||
    fail "Could not stop VM after failed first attempt"
fi

if ! wait_for_vm_stopped; then
```

```
  fail "VM $VMID did not stop within 30 seconds"
fi

log "VM $VMID stopped after first attempt"

sleep 2

verify_vfio
wake_d500

if start_and_verify 2; then
  log "SUCCESS: VM $VMID started with D500 on second attempt"
  exit 0
fi

log "Second attempt failed D500 QMP verification"

if qm status "$VMID" 2>/dev/null | grep -q "status: running"; then
  log "Stopping VM $VMID after second failed attempt"
  qm stop "$VMID" || true
fi

fail "D500 startup failed after two attempts; manual investigation required"
```

Install and validate:

```
chmod 755 /usr/local/sbin/d500-jellyfin-start
bash -n /usr/local/sbin/d500-jellyfin-start
echo $?
# Expected: 0
```

9. systemd Startup Service

Create /etc/systemd/system/d500-jellyfin.service:

```
[Unit]
Description=Start Jellyfin VM with Apple FirePro D500 passthrough
After=pve-guests.service
Requires=pve-guests.service

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/d500-jellyfin-start
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

Reload systemd and enable the service:

```
systemctl daemon-reload
systemctl enable d500-jellyfin.service
```

Give the helper exclusive ownership of automatic VM startup

```
qm set 103 --onboot 0
qm config 103 | grep -E '^(onboot|startup):'
```

The tested VM retained startup metadata but onboot was disabled:

```
onboot: 0
startup: order=10,up=10
```

WARNING: *Disable any older experimental service that also wakes the GPU or starts the same VM. Two independent startup paths can race each other.*

Why After=pve-guests.service matters

Earlier startup attempts ran before Proxmox cluster/guest-management state was ready and produced “cluster not ready - no quorum?” errors. Ordering the helper after pve-guests.service solved that timing problem on this host. Adding graphical.target to the ordering later created an undesirable dependency/order relationship and prevented the service from running, so it was removed.

10. Preflight and Cold-Boot Test

```
echo "=== SERVICES ==="
systemctl is-enabled d500-jellyfin.service

echo "=== VM ONBOOT ==="
qm config 103 | grep -E '^(onboot|startup):'

echo "=== SCRIPT SYNTAX ==="
bash -n /usr/local/sbin/d500-jellyfin-start
echo "syntax exit code: $?"
```

For the real validation, power the host completely off, then power it back on. Do not manually wake the GPU or start the VM. After boot, inspect:

```
qm status 103
systemctl status d500-jellyfin.service --no-pager
journalctl -b -u d500-jellyfin.service --no-pager
```

11. Successful Unattended Cold-Boot Result

The final test was a true power-off/power-on cycle. The service ran automatically. Attempt 1 initially passed, then failed five seconds later. V3 detected the delayed dropout, stopped the bad VM, reclaimed and woke the D500, and launched attempt 2. All six attempt-2 checks passed.

```
16:49:49  D500 stabilization check 1/6 PASS
16:49:54  D500 stabilization check 2/6 FAIL
        First attempt did not acquire D500; stopping VM 103
        ...
16:50:01  Drivers: GPU=vfio-pci AUDIO=vfio-pci
16:50:03  Power state after wake: GPU=D0 AUDIO=D0
16:50:03  Starting VM 103: attempt 2
16:50:15  D500 stabilization check 1/6 PASS
16:50:20  D500 stabilization check 2/6 PASS
16:50:26  D500 stabilization check 3/6 PASS
16:50:31  D500 stabilization check 4/6 PASS
16:50:36  D500 stabilization check 5/6 PASS
16:50:41  D500 stabilization check 6/6 PASS
16:50:41  D500 remained attached for 30 seconds on attempt 2
16:50:41  SUCCESS: VM 103 started with D500 on second attempt
```

NOTE: *During the failed first attempt, Proxmox also logged “no PCI device found for 06:00” while stopping the VM. This matched the previously observed physical disappearance/re-enumeration behavior.*

12. Validate the Guest, Not Just QEMU

Once the helper reports success, verify that Ubuntu actually initialized the GPU. With QEMU Guest Agent available:

```
qm guest exec 103 -- bash -lc '
echo "=== D500 PCI ==="
lspci -nnk | grep -A3 -E "VGA|Audio"

echo "=== DRI ==="
ls -l /dev/dri

echo "=== RADEON INIT ==="
dmesg | grep -Ei "ATOM BIOS|UVD initialized|VCE initialized" | tail -10
'
```

The validated guest reported:

```
01:00.0 VGA ... Tahiti LE [1002:679e]
   Kernel driver in use: radeon
01:00.1 Audio ... Tahiti HDMI Audio [1002:aaa0]
   Kernel driver in use: snd_hda_intel

/dev/dri/card0
/dev/dri/card1
/dev/dri/renderD128

ATOM BIOS: Tahiti
[drm] UVD initialized successfully.
[drm] VCE initialized successfully.
```

This is stronger evidence than “the PCI device exists.” It shows that the guest driver initialized the Tahiti GPU and exposed a render node. Jellyfin codec support and actual transcoding should still be tested separately.

13. Troubleshooting Matrix

Symptom	Likely meaning	Recommended check/action
vfio-pci bound, but D3hot	Correct ownership, low-power device	Wake GPU and audio to D0 before assignment.
reset: Inappropriate ioctl for device	D500 reset limitation	Do not assume fatal failure; verify actual QEMU assignment.
qm start succeeds, guest has no AMD GPU	QEMU process success != passthrough success	Inspect QMP/info pci; use sustained verification.
QMP passes once, guest later lacks GPU	Delayed dropout	Use V3 30-second stabilization loop.
Bus 1 bridge exists but empty	D500 fell out of QEMU	Stop failed VM, settle, verify vfio-pci, wake, retry.
no PCI device found for 06:00 during failed stop	Device temporarily disappeared	Allow re-enumeration; do not force raw bridge reset.
vfio_pci unknown parameter	Malformed ids list	Remove spaces after commas

messages		and rebuild initramfs.
cluster not ready - no quorum? during boot helper	Helper started too early	Order after pve-guests.service.
Black console after GRUB	Not necessarily a failed guest	Check VM status, QMP, guest agent, and guest PCI state before intervening.
PCI ROM read returns 0x0000 / I/O error	Apple ROM not exposed normally	Use ACPI VFCT from your own hardware.

14. Approaches That Did Not Solve It

- Waking the D500 to D0 without verifying QEMU.
- Trusting qm start as proof of successful passthrough.
- Checking QMP only once five seconds after startup.
- Attempting the advertised direct GPU reset after it returned an ioctl error.
- Looking for a usable reset node on the upstream root port.
- Reading the VBIOS through the ordinary PCI expansion ROM sysfs path.
- Using a superficially similar Internet ROM that did not match the D500 device/board identity.
- Starting the VM before Proxmox guest/cluster services were ready.
- Adding unnecessary systemd ordering dependencies such as graphical.target.

15. Limitations and Maintenance

- This workaround does not repair the underlying Apple/AMD reset behavior. It detects a bad first assignment and recovers from it.
- The 30-second window was chosen from observed failures. It is not proof that a device can never fail later.
- The helper assumes the D500 is dedicated to this VM during startup.
- The helper retries only once. Two failed attempts intentionally stop and require manual investigation.
- QMP verification proves sustained PCI attachment, not complete guest driver health. Guest validation remains important.
- Kernel, QEMU, Proxmox, or firmware upgrades can change behavior. Re-test cold boot after major upgrades.
- The D500 is an older Tahiti GPU. Successful UVD/VCE initialization does not imply support for modern codecs such as AV1 or every HEVC profile.

16. Recommended GitHub Repository Layout

```
macpro6-1-d500-proxmox/
├── README.md
├── docs/
│   ├── FirePro-D500-Proxmox-Guide.pdf
│   └── FirePro-D500-Proxmox-Guide.docx
├── scripts/
```

```
│   ├── d500-jellyfin-start
│   └── extract-d500-vfct.py       # optional companion utility
├── systemd/
│   └── d500-jellyfin.service
└── examples/
    ├── vfio.conf.example
    └── vm-config.example
```

WARNING: *Do not commit *.rom or VFCT.bin unless you have separately established that you have the right to redistribute those firmware binaries. The safer repository design is to provide extraction/validation tooling and have users obtain firmware from their own Mac Pro.*

17. Known-Good VM Configuration

```
agent: 1
boot: order=scsi0;ide2;net0
cores: 6
cpu: host
hostpci0: 06:00,pcie=1,romfile=D500-LB-029.rom
ide2: local:iso/ubuntu-24.04.5-live-server-amd64.iso,media=cdrom,size=3984850K
machine: q35
memory: 8424
name: Jellyfin
net0: virtio=BC:24:11:1A:5B:7F,bridge=vmbr0,firewall=1
numa: 0
onboot: 0
startup: order=10,up=10
ostype: l26
scsi0: local-lvm:vm-103-disk-0,iothread=1,size=100G
scsihw: virtio-scsi-single
sockets: 1
```

NOTE: *This is a reproducibility record, not a universal VM template. Adapt storage, networking, CPU, RAM, PCI addresses, and ROM filename to your system.*

18. Conclusion

The difficult part of this problem was not getting a D500 to appear in a VM once. It was proving why a seemingly successful cold-boot assignment could later collapse, then designing a startup path that detects the actual failure rather than trusting process state.

On the validated Mac Pro 6,1, the successful pattern is: early vfio-pci ownership, wake both D500 functions to D0, provide the correct Apple VBIOS, start QEMU, verify both QEMU PCI functions continuously for 30 seconds, and retry once after a failed first assignment. The final unattended cold-boot test reproduced the delayed dropout and recovered automatically on attempt 2. Ubuntu then initialized the D500 with the radeon driver, exposed renderD128, and initialized UVD and VCE.

The practical lesson: a running VM is not proof of successful GPU passthrough. Verify the hardware, verify it stays attached, and recover from the failure you actually observe.

Appendix A - Quick Command Reference

```
# Host PCI inventory
lspci -nnk | grep -A4 -Ei 'VGA|Display|Audio'

# D500 power
cat /sys/bus/pci/devices/0000:06:00.0/power_state
cat /sys/bus/pci/devices/0000:06:00.1/power_state

# Wake D500
echo on > /sys/bus/pci/devices/0000:06:00.0/power/control
echo on > /sys/bus/pci/devices/0000:06:00.1/power/control

# Verify VM and service
qm status 103
systemctl status d500-jellyfin.service --no-pager
journalctl -b -u d500-jellyfin.service --no-pager

# Inspect generated QEMU command
qm showcmd 103 --pretty

# Interactive QEMU PCI inspection
qm monitor 103
# then: info pci
```

Appendix B - Reference Firmware Fingerprints from the Test Host

Artifact	Size	SHA-256
VFCT.bin	131232 bytes	267ae5cd39050b2ffdd0d5911c3e860aa8fb5dce745c7dfc2e9aecb367fe65a8
D500-LA-029.rom	65536 bytes	eb2aec73a30e93aa3867a34e4eb2eeca01c421780fff3dd5ca727ffa53227a99
D500-LB-029.rom	65536 bytes	c07040282180be27bc795c8a8e4f579b9dd3bee20ef3cd0dc025d7fec9675d31

NOTE: *These fingerprints document the test system. They are not a download manifest and are not a requirement that another Mac Pro produce identical hashes.*